

tests/letsencrypt/ cert_analyzer_selftest.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:**

Active §11.4.107(10) self-validated-analyzer TAP self-test for
cert_analyzer.sh. Hermetic, offline, deterministic.

Companion (§11.4.18) to the in-source documentation block
at the top of the script.

Overview / Purpose

Proves every cert_analyzer.sh function is CORRECT against the golden-GOOD fixtures AND provably REJECTS every golden-BAD fixture — the §11.4.107(10) self-validated-analyzer discipline (an analyzer that ACCEPTS its golden-bad fixture is a bluff gate). Golden-bad cases covered: expired / not-yet-valid / expired-just-now & NotBefore/NotAfter boundaries / wrong-CA / wrong-host / near-expiry / substring-not-a-match / double-wildcard SAN / empty-SAN (no CN fallback) / IP-only SAN / malformed-truncated PEM.

Fully hermetic (no network, no ACME, no container boot); deterministic via a pinned reference "now" of 2026-07-01T12:00:00Z (§11.4.50). The SAME good leaf is asserted valid at a mid-window now AND expired at a past-NotAfter now — via both the positional-arg seam and the CERT_ANALYZER_NOW_EPOCH env seam — with no time-travel.

Usage

```
bash tests/letsencrypt/cert_analyzer_selftest.sh
```

No arguments.

Inputs

- None on the command line.

- Sources the library under test (`cert_analyzer.sh`) and consumes the fixtures under `tests/letsencrypt/fixtures/`. If `good_leaf.pem` is missing/empty it self-bootstraps by running `gen_fixtures.sh` (§11.4.77).

Outputs

- TAP (Test Anything Protocol) on stdout — one ok/not ok line per case, a 1..N plan, a # tests=... passed=... failed=... line, and a # RESULT: line.
- A copy of the TAP stream written to `qa-results/letsencrypt/cert-analyzer/<run-id>/selftest.tap` (`run-id = UTC YYYYMMDDTHHMMSSZ`); the artefact path is echoed to stderr.
- Exit code: 0 iff ALL assertions pass (zero not ok); 1 if any fail.

Side-effects

- Creates `qa-results/letsencrypt/cert-analyzer/<run-id>/` and writes `selftest.tap` there (gitignored evidence dir).
- May write the fixture corpus on first run (bootstrap via `gen_fixtures.sh`).
- No network, no ACME, no containers.

Dependencies

bash (body is POSIX-clean, parses under `sh -n`), `openssl`, GNU `date -d`, `awk/sed/tr`, `mkdir`, `tee`. Fixtures self-bootstrap when absent.

Edge cases

- **Fixtures absent** → bootstrapped from `gen_fixtures.sh` before the run.
- **tee subshell** → the pass/fail total is re-derived from the artefact (`grep -q '^not ok '`) so the exit code is authoritative even though `$FAILS` is not visible outside the tee'd pipeline.
- Distinguishes an ABSENT PEM (rc 2) from a PRESENT-but-malformed PEM (also rc 2) — both are asserted as no-false-PASS cases.
- Value-equality checks pin `cert_days_remaining` scalars (`good=59`, `near-expiry=4`, `expired=-122`) against the pinned now.

Related scripts

- `tests/letsencrypt/cert_analyzer.sh` — the library under test.

- tests/letsencrypt/fixtures/gen_fixtures.sh — the golden fixture generator (bootstrap + regeneration, §11.4.77).
- docs/scripts/cert_analyzer_selfvalidation_test.md — the sibling self-validation gate doc.
- Constitution §11.4.107(10) / §11.4.50 / §11.4.69 / §1.1; design LETSENCRYPT_HTTPS.md §6 + LETSENCRYPT_HTTPS_PLAN.md Phase 3.

Last verified

2026-07-01 — documented against the script source; body parses under `sh -n`. Runs hermetically (no network, no ACME) and emits its TAP artefact under `qa-results/letsencrypt/cert-analyzer/`.